Java Silver（SE 17・1Z0-825-JPN）　4 クラスの定義とインスタンス　#08

クラスとオブジェクト

変数に入っているのは参照。new で作り、コンストラクタで整え、参照がなくなれば片づけられる

各ページの下に、根拠にした Oracle の資料（JLS 4・6・8・12・15 章、Java Tutorials、Java SE 17 API）を書いた（青い文字を押すと開く）。コードは Claude が作り、JDK 17 で動かして確かめた

全体像 試験内容チェックリスト 4（クラスの宣言とインスタンス化、ライフサイクル、メソッド、オーバーロード、コンストラクタ）

1 クラスとインスタンス
フィールドの既定値、参照のコピー、null

2 メソッド
戻り値と return、値渡し、可変長引数

3 オーバーロード
シグネチャのちがい、3 段階の解決、あいまいな呼び出し

4 コンストラクタと初期化
デフォルトコンストラクタ、this(…)、this、初期化の順番

5 ライフサイクル
参照がなくなると GC の対象。System.gc() はすすめるだけ

1 つのクラスの中身（Claude が作成）

```
class Counter {
  int count;                    // フィールド（既定値 0）
  { System.out.print("init "); }  // インスタンス初期化子
  Counter() { }                   // コンストラクタ（戻り値の型なし）
  Counter(int count) {            // オーバーロード
    this.count = count;         // this でフィールドを指す
  }
  void inc() { count++; }       // メソッド
  int get() { return count; }
}
```

static は #09、継承と super(…) は #11。この PDF はインスタンス（static のないもの）を扱う。

出典：Oracle「Java SE 17 Programmer I（1Z0-825-JPN）」試験内容チェックリスト／JLS 8 Classes

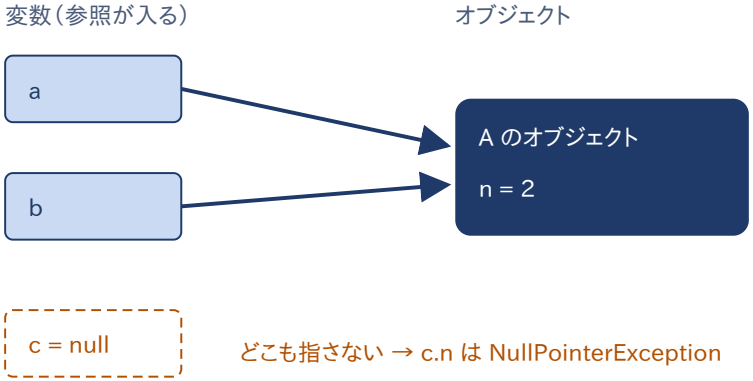
第1章

クラスとインスタンス

クラスは設計図。変数に入るのはオブジェクトへの参照。

変数とオブジェクトの関係

A b = a; のあと（Claude が作成）




```
A a = new A();  a.n = 1;
A b = a;              // 参照のコピー
b.n = 2;
System.out.println(a.n + " " + (a == b));   // 2 true
```

コード	結果（JDK 17）
`A b = a;` のあと `b.n = 2`	a.n も 2
`new A(1) == new A(1)`	false（別のオブジェクト）
`a.equals(b)`（A は equals を書いていない）	== と同じ
C の a・b で a.inc() 2 回、b.inc() 1 回	a.count 2、b.count 1
`A a = null; a.n`	NullPointerException（実行時）

インスタンス変数はオブジェクトごと。new するたびに、そのオブジェクト用のフィールドが新しくできる。

出典：JLS 4.3.1 Objects（The reference values (often just references) are pointers to these objects, and a special null reference, which refers to no object）／JLS 8.3.1.1（a new variable associated with that instance is created for every instance variable）／API Object.equals（x and y refer to the same object (x == y has the value true)）／結果は JDK 17 で確かめたもの

既定値があるのはフィールドだけ

フィールドの型	既定値
byte・short・int・long	0
float・double	0.0
char	'\u0000'（int にすると 0）
boolean	false
参照型（String・配列・クラス）	null

```
class A { int i; double d; boolean b; String s; char c; long[] arr; }
// a.i a.d a.b a.s (int)a.c a.arr を表示
```

```
0 0.0 false null 0 null
```

ローカル変数には既定値がない（Claude が作成）

```
int x;
System.out.println(x);
```

```
error: variable x might not have been initialized
```

	フィールド	ローカル変数
書く場所	クラスの中・メソッドの外	メソッド・ブロックの中
既定値	あり	なし
代入前に使う	既定値が読める	コンパイルエラー
配列の要素	どこで作っても既定値が入る（#04）	

出典：JLS 4.12.5 Initial Values of Variables（Each class variable, instance variable, or array component is initialized with a default value when it is created／A local variable … must be explicitly given a value before it is used）／結果は JDK 17 で確かめたもの

第2章

メソッド

引数はいつも値のコピー。参照型なら参照のコピー。

メソッドの形と return

public	int	twice	(int x)	{ return x*2; }
修飾子	戻り値の型	名前	引数の並び	本体

名前＋引数の型の並び ＝ シグネチャ（戻り値の型は含まない）

書き方	結果（JDK 17）
`int f(int x) { if (x > 0) return 1; }`	missing return statement
`void f() { return 1; }`	incompatible types: unexpected return value
`void f() { return; }`	OK（途中で終わる）
戻り値を使わずに `twice(3);`	OK（値は捨てられる）

this を返すとつなげて呼べる（Claude が作成）

```
class C {
  int v;
  C add(int x) { v += x; return this; }
}
C c = new C().add(1).add(2);
System.out.println(c.v);    // 3
```

戻り値の型があるメソッドは、どの道を通っても `return 値;` で終わる。
「最後まで来てしまう道」が 1 本でもあればコンパイルエラー。

void は値を返さない。`return;`（値なし）は書ける。

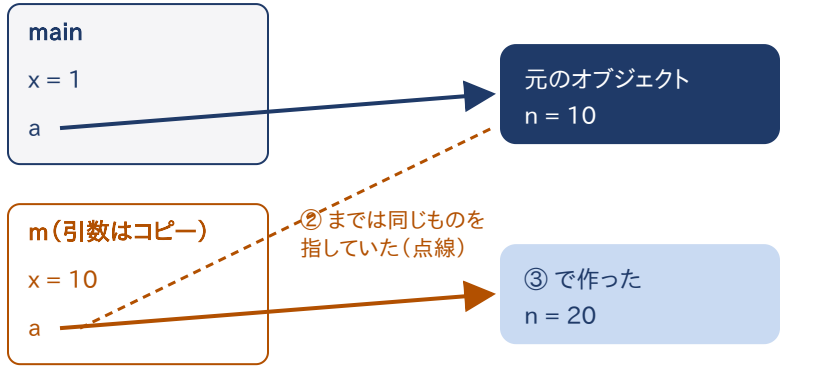
出典：JLS 8.4.2 Method Signature・8.4.7 Method Body（a compile-time error occurs if the body of the method can complete normally）／JLS 14.17（the return target of a return statement with value Expression is either a constructor, or a method that is declared void）／結果は JDK 17 で確かめたもの

引数は値渡し ー 参照型は参照のコピー

```
static void m(int x, A a) {
  x = 10;          // ① コピーの x だけ変わる
  a.n = 10;        // ② 同じオブジェクトを変える
  a = new A();     // ③ m の a が別を指すだけ
  a.n = 20;
}
// main：int x = 1; A a = new A(); m(x, a);
System.out.println(x + " " + a.n);   // 1 10
```

メソッドの中で	呼び出し元から見ると
基本型の引数に代入	**変わらない**
引数のオブジェクトのフィールドを変える	**変わる**
引数に別のオブジェクトを代入	**変わらない**
String の引数に `s = s + "x"`	**変わらない**
StringBuilder の引数に `sb.append("x")`	**変わる**

③ の直後（Claude が作成）



Java に参照渡しはない。参照型でも「参照という値」をコピーして渡している。

出典：JLS 8.4.1 Formal Parameters（the values of the actual argument expressions initialize newly created parameter variables, each of the declared type, before execution of the body of the method or constructor）／API String（Strings are constant）／結果は JDK 17 で確かめたもの

可変長引数

```
static int sum(int... v) {   // 中では int[]
  int s = 0;
  for (int x : v) s += x;
  return s + v.length * 100;
}
sum()                    // 0    （長さ 0 の配列）
sum(1, 2, 3)             // 306  （長さ 3）
sum(new int[] {4, 5})    // 209  （配列をそのまま）
```

引数なしで呼んでも null ではない。長さ 0 の配列が渡るので、v.length も拡張 for もそのまま使える。

宣言	結果（JDK 17）
`m(String s, int... a)`	OK
`m(int... a, String s)`	varargs parameter must be the last parameter
`m(int... a, int... b)`	同じエラー（1 つまで）
`m(int a...)`	',', ')', or '[' expected
`m(int[]... a)`	OK（中では int[][]）
`m(String... a)` を `m()`	OK（a.length は 0）

... は型のすぐあと（空白をはさむこともできるが、すすめられない）。main を `public static void main(String... args)` と書くこともできる（#01）。

出典：JLS 8.4.1（At most one variable arity parameter is permitted for a method or constructor. It is a compile-time error if a variable arity parameter appears anywhere in the list of parameter specifiers except the last position）／結果は JDK 17 で確かめたもの

第3章

オーバーロード

同じ名前、ちがう引数。どれを呼ぶかはコンパイル時に 3 段階で決まる。

オーバーロードになるもの・ならないもの

同じクラスに 2 つ書く	結果（JDK 17）
`f(int)` と `f(String)`	OK（型がちがう）
`f(int)` と `f(int, int)`	OK（数がちがう）
`f(int, long)` と `f(long, int)`	OK（順番がちがう）
`int f(int x)` と `long f(int y)`	method f(int) is already defined
`A() { }` を 2 つ	constructor A() is already defined

シグネチャ＝名前＋引数の型の並び。戻り値の型・引数の名前・修飾子・throws はちがっても区別にならない。

呼び出しのあいまいさ（JDK 17）

```
static void m(int a, long b) { … }
static void m(long a, int b) { … }
m(1, 2);   // どちらにも拡大で合う
```

```
error: reference to m is ambiguous
```

宣言は OK でも、呼び出し方によってはコンパイルエラーになる。m(1, 2L) なら m(int, long) に決まる。

オーバーライドとはちがう。オーバーロードは同じ名前で引数がちがうメソッドを並べること。オーバーライドは親クラスのメソッドを同じシグネチャで書き直すこと（#11）。

出典：JLS 8.4.9 Overloading（have the same name but signatures that are not override-equivalent, then the method name is said to be overloaded）・8.4.2（It is a compile-time error to declare two methods with override-equivalent signatures in a class）／結果は JDK 17 で確かめたもの

どれが呼ばれるか ― 3 段階

m(5) の探し方（Claude が作成）

1 段階目：型の拡大だけ
int → long → float → double など

見つかれば
ここで決まる

2 段階目：ボクシングも許す
int → Integer（→ Object など）

1 で無ければ

3 段階目：可変長引数も許す
int... など

2 で無ければ

候補（呼び出しは m(5)）	結果（JDK 17）
m(long)・m(Integer)・m(int...)	long
m(Integer)・m(int...)	Integer
m(Long)・m(int...)	int...
m(Long) だけ	int cannot be converted to Long
m(double)・m(Object) に m('a')	double 97.0
m(int, int)・m(int...) に m(1, 2)	m(int, int)
同じ 2 つに m(1, 2, 3)・m()	m(int...)

「拡大してからボクシング」はできない。int は Integer にしかボクシングされないので、int → Long はどの段階でも合わない。

出典：JLS 15.12.2 Compile-Time Step 2（The first phase performs overload resolution without permitting boxing or unboxing conversion, or the use of variable arity method invocation／The second phase … allowing boxing and unboxing／The third phase allows overloading to be combined with variable arity methods）／JLS 5.3 Invocation Contexts／結果は JDK 17 で確かめたもの

最も具体的なメソッドと null

```
static void m(Object o) { … "Object" }
static void m(String s) { … "String" }

m(null);          // String
m("a");           // String
Object o = "b";
m(o);             // Object（宣言の型で決まる）
```

Object

String は Object の一種

String

→ m(String) の方が具体的

候補に m(null)	結果（JDK 17）
m(Object)・m(String)	String
m(Integer)・m(String)	reference to m is ambiguous
m(Integer)・m(String) に `m((String) null)`	String（キャストで決まる）

「より具体的」＝ 一方の引数の型が、もう一方の引数の型の一種。Integer と String はどちらも相手の一種ではないので、どちらにも決められない。

どれを呼ぶかはコンパイル時。実行時のオブジェクトの型ではなく、引数の式の型で選ぶ。

出典：JLS 15.12.2.5 Choosing the Most Specific Method（the most specific method is chosen／It is possible that no method is the most specific, because there are two or more methods that are maximally specific／Otherwise, the method invocation is ambiguous, and a compile-time error occurs）／結果は JDK 17 で確かめたもの

第4章

コンストラクタと初期化

戻り値の型を書かない。1 つでも書けば自動では入らない。

コンストラクタとデフォルトコンストラクタ

クラスの書き方	new A() は
コンストラクタを 1 つも書かない	OK（デフォルトコンストラクタ）
`A(int x)` だけ書く	constructor A in class A cannot be applied to given types
`A()` と `A(int x)` を書く	OK
`void A() { }` だけ書く	OK（それはメソッド。デフォルトコンストラクタが入る）

コンパイラが入れるもの（JLS の例を Claude が書き直した）

```
public class Point { int x, y; }
// ↓ と同じ
public class Point { int x, y; public Point() { super(); } }
```

デフォルトコンストラクタ：引数なし。アクセス修飾子はクラスと同じ（クラスに修飾子がなければパッケージアクセス）。本体は親クラスの引数なしコンストラクタを呼ぶだけ（#11）。

戻り値の型を書くとメソッド（Claude が作成）

```
class A {
  void A() { System.out.print("method "); }
}
A a = new A();              // 何も表示しない
System.out.print("created ");
a.A();                      // method
```

```
created method
```

出典：JLS 8.8 Constructor Declarations（In all other respects, a constructor declaration looks just like a method declaration that has no result／Constructor declarations are not members）・8.8.9 Default Constructor（If a class contains no constructor declarations, then a default constructor is implicitly declared／例 8.8.9-1）／結果は JDK 17 で確かめたもの

this(…) と this

```
class A {
  A() {
    this(5);                    // 必ず最初の文
    System.out.print("A() ");
  }
  A(int x) { System.out.print("A(" + x + ") "); }
}
new A();
```

```
A(5) A()
```

書き方	結果（JDK 17）
print のあとに `this(1);`	call to this must be first statement in constructor
A() → this(1)、A(int) → this()	recursive constructor invocation

```
class A { int n; A(int n) { n = n; } }        // 0
class B { int n; B(int n) { this.n = n; } }   // 5
// new A(5).n と new B(5).n
```

```
0 5
```

シャドーイング：引数 n がフィールド n を隠す。ただの n は引数。フィールドは this.n。

this ＝ 今のオブジェクトへの参照。インスタンスメソッドとコンストラクタの中で使える。static メソッドの中では使えない（#09）。

メソッドの中でも、フィールド n と同じ名前のローカル変数 n を宣言できる。`n + " " + this.n` は "1 5" のようになる。

出典：JLS 8.8.7 Constructor Body（The first statement of a constructor body may be an explicit invocation of another constructor／It is a compile-time error for a constructor to directly or indirectly invoke itself）／JLS 6.4.1 Shadowing／JLS 15.8.3 this／結果は JDK 17 で確かめたもの

初期化の順番と final フィールド

```
class A {
  int x = p("f1 ");       // ①
  { p("i1 "); }            // ②
  A() { p("c "); }         // ⑤ 本体は最後
  { p("i2 "); }            // ③
  int y = p("f2 ");       // ④
}
new A();  new A();
```

```
f1 i1 i2 f2 c
f1 i1 i2 f2 c
```

new のたびに全部やり直す。`int x = 10;`・`{ x = 30; }`・`A() { x = 20; }` なら x は 20。

new A(…) のとき（Claude が作成）

1 メモリを確保し、フィールドを既定値にする

2 this(…) があれば、そのコンストラクタへ

3 フィールドの初期化式と初期化子を、上から順に

4 コンストラクタの本体の残り

final フィールド	結果（JDK 17）
`final int x;` だけ	variable x not initialized in the default constructor
A() で代入、A(int) で代入しない	variable x might not have been initialized

出典：JLS 12.5 Creation of New Class Instances（Execute the instance initializers and instance variable initializers for this class … in the left-to-right order in which they appear textually ／Execute the rest of the body of this constructor）／JLS 8.6・8.3.1.2（A blank final instance variable must be definitely assigned … at the end of every constructor）／結果は JDK 17 で確かめたもの

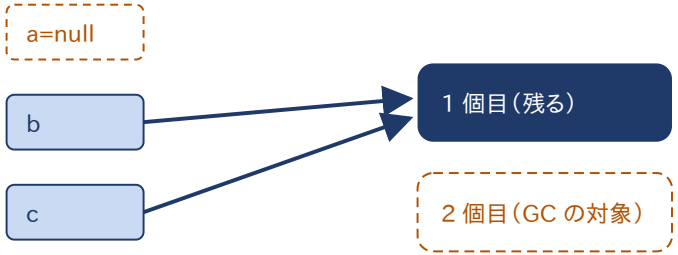
第5章

オブジェクトのライフサイクル

new で生まれ、どこからもたどれなくなると片づけの対象になる。

ガベージコレクションの対象を数える

```
A a = new A();   // 1 個目
A b = new A();   // 2 個目
A c = a;
a = null;
b = c;
// here
```



参照がなくなる場面	例
null を代入	`a = null;`
別のオブジェクトを代入	`b = c;`
スコープを出る	メソッドが終わってローカル変数が消える

> **対象になる ≠ すぐ消える。**いつ実際に解放するかは JVM が決める。Java にはオブジェクトを消す命令（delete）がない。

> **System.gc()** は GC を**すすめる**（suggests）だけ。実行は保証されない。

上のコードは JDK 17 で WeakReference を使い、System.gc() 後に 2 個目だけが回収されたことを確かめた（回収のタイミングは環境で変わりうる）。

出典：The Java Tutorials「Using Objects」（An object is eligible for garbage collection when there are no more references to that object）／API System.gc（Calling the gc method suggests that the Java Virtual Machine expend effort toward recycling unused objects）／JLS 12.6（A reachable object is any object that can be accessed in any potential continuing computation from any live thread）

問われ方とまちがえやすい点

こう聞かれたら	答えの考え方
フィールドの表示結果	既定値（0・0.0・false・null）。ローカル変数なら未初期化でコンパイルエラー
代入やメソッドのあとの値	参照のコピーなら同じオブジェクト。引数への代入は呼び出し元に戻らない
どのオーバーロードが呼ばれるか	拡大 → ボクシング → 可変長の順。同じ段階なら具体的な方。決まらなければ ambiguous
コンパイルできるか（メソッド）	戻り値の型だけちがう、missing return、void の return 値、可変長は最後に 1 つ
コンパイルできるか（コンストラクタ）	1 つでも書けばデフォルトなし、this(…) は先頭、final フィールドは必ず代入
表示の順番	初期化式・初期化子は上から、コンストラクタ本体は最後。this(…) の先が先
GC の対象はいくつ	矢印の図を書き、どの変数からもたどれないオブジェクトを数える

出典：このPDFの各ページの資料

まとめ

クラスとインスタンス
フィールドは既定値あり。変数には参照が入る

メソッド
値渡し。可変長引数は最後に 1 つ、中は配列

オーバーロード
引数の型・数・順番。3 段階で解決

コンストラクタ
戻り値の型なし。this(…) は先頭。本体は初期化子のあと

ライフサイクル
参照がなくなると GC の対象

こんなとき	結果
A(int) だけのクラスで new A()	コンパイルエラー
m(long)・m(Integer) に m(5)	m(long)
m(Integer)・m(String) に m(null)	ambiguous
コンストラクタで `n = n;`	フィールドは変わらない
System.gc()	すすめるだけ

次は #09 static とアクセス修飾子。クラス変数・クラスメソッド、static の初期化子、public・protected・private とカプセル化へ進む。

出典：このPDFの各ページの資料